Unlearning a group from an in-context prompt: how much does corrupting it actually remove?

Abstract

We test whether a group of training examples can be unlearned from a linear-attention in-context learner by corrupting that group inside the prompt, without changing any weights. The question is whether a membership adversary is actually defeated or only blinded. We sweep six corruption families over four decades of strength, on a synthetic Gaussian task and on MNIST features, across nine seed combinations.

The membership signal sits in the first moment of the residual. Additive noise cannot reach it. It moves the adversary toward chance by inflating variance instead, and a shared-noise control attributes 84–100% of that movement to masking rather than removal. Reaching chance this way costs 762–1699× in preservation error on the synthetic task and 4375–7838× on MNIST.

Corruptions that move the mean, such as label flips, reach chance 6× (synthetic) to 16–21× (MNIST) more cheaply. We derive the flip's effect on the read-out exactly. This makes membership AUC a differentiable function of the flip probability and gives $\theta^* = (N+1)\delta_0/2\sum_i a_i$ for the optimal flip. Optimising it reaches AUC $= 0.4987 \pm 0.0005$ on MNIST at $\varepsilon \approx 0.05$, with the same value on every seed. The closed form agrees with the REINFORCE estimator and runs 7–9× faster. On one synthetic architecture the required flip fraction is above one, so no label flip can cancel the gap at all; there the optimiser saturates against that bound instead of converging.

1 Problem

Claim. In this model the membership signal sits in the first moment of the residual. A corruption erases it only if it moves the mean. A variance-based corruption can mask the signal but cannot reach it, and pays for the masking in preservation error. The best corruption is the one that cancels the baseline mean gap. We derive it in closed form, and it predicts the learned flip probability wherever that cancellation is possible.

A model is shown a prompt containing examples from three groups and must predict the query. One group is to be forgotten. We do not touch the weights. We corrupt that group's tokens in the prompt and ask whether a membership adversary can still tell the group was trained on.

The results below are evidence for the claim above. The negative ones show where the signal is not, which is what makes the positive one interpretable.

Two quantities trade off against each other.

Removal. An adversary sees the sign-aligned residual $r = \mathrm{sign}(y)(\hat{y} - y)$ at a fixed probe point and must decide which of two models produced it: M_{full}, trained on all three groups, or M_{orac}, retrained without the forget group. We report the Mann–Whitney statistic

$$\mathrm{AUC} = \Pr\big[\, r(M_{\mathrm{full}}) > r(M_{\mathrm{orac}}) \,\big] + \tfrac{1}{2}\Pr\big[\, r(M_{\mathrm{full}}) = r(M_{\mathrm{orac}}) \,\big], \tag{1}$$

so AUC $= \frac{1}{2}$ means the adversary is guessing. *Both models are scored on the same corrupted prompt with the same noise draw*, so detecting the edit cannot be mistaken for detecting membership.

The score is oriented by the usual membership-inference convention: the *member* hypothesis scores higher, so a working attack reads AUC $> \frac{1}{2}$ and chance is $\frac{1}{2}$. Since the full model has seen the forget group it fits forget-group queries *better* and its sign-aligned residual is smaller, the membership score is the negated residual, $-r$. The ordering is fixed in the code rather than chosen per run, so it cannot be tuned after the fact. We do not symmetrise: $\max(a, 1-a)$ is biased upward under the null (it reads ≈ 0.54 when the truth is 0.50), which would manufacture signal at exactly the point these experiments measure.

Preservation. How far the corrupted model has drifted from the retrain oracle it is meant to imitate:

$$\varepsilon = \mathrm{KL}\big(p_{\text{oracle}} \,\|\, p_{\text{unlearned}}\big) \tag{2}$$

on the forget-population residual law, which is fit as a univariate Gaussian (μ, v) over the probe population, so ε is a Gaussian KL. Unlearning is only useful if AUC $\to \frac{1}{2}$ *and* ε stays small.

Are AUC and ε the same measurement? Both depend on the first two moments, so the cost ratios below could be tautological. They are not. AUC depends only on the standardised gap $(\mu_1 - \mu_0)/\sqrt{v_1 + v_0}$ and is invariant to rescaling both populations together. ε is not scale-invariant and grows with the variance ratio. So a corruption that inflates v drives AUC to $\frac{1}{2}$ while ε keeps climbing. That is the masking regime, and it is why the two can be traded against each other.

Corruptions. Applied to the forget tokens (x_f, y_f) only:

name	edit	parameter	channel
C1	$y_f \to y_f + \epsilon,\ \ \epsilon \sim \mathcal{N}(0, \sigma^2)$	σ^2	variance
C2	$x_f \to x_f + \epsilon,\ \ \epsilon \sim \mathcal{N}(0, \sigma^2 I)$	σ^2	variance
C3	both, independent draws	σ^2	variance
flip	$y_f \to (1 - 2t)\, y_f$	t	mean
bern	$y_f \to (1 - 2B)\, y_f,\ \ B \sim \mathrm{Bern}(\theta)$	θ	mean + variance
whiten	$x_f \to$ covariance-matched to retain groups	mixing	—

The last two matter most. `flip` has a mean shift and no variance. `bern` has the same mean, $\mathbb{E}[(1 - 2B)y] = (1 - 2\theta)y$, plus variance $4\theta(1-\theta)y^2$. Write the predicted AUC as $\Phi\big((\mu_1 - \mu_0)/\sqrt{v_1 + v_0}\big)$. A zero-mean corruption can only inflate the denominator, so it can never push AUC through $\frac{1}{2}$. A mean shift moves the numerator directly.

2 Method

Algorithm 1 — measuring removal and preservation

Input: group spec, forget group f, corruption mode and grid

1. Train two shadow ensembles of $S = 512$ linear-attention models: M_{full} on all groups, M_{orac} on the retain groups only.
2. Draw a probe of $P = 64$ prompts and *freeze* it.
3. For each corruption strength t in the grid:
  (a) corrupt the forget tokens of the probe $\to \tilde{X}$;
  (b) run *both* ensembles on the same $\tilde{X}$;
  (c) AUC(t) from the two residual populations;
  (d) $\varepsilon(t)$ from the fitted residual laws;
  (e) *shared-noise control*: repeat (a)–(c) broadcasting one noise draw across all S shadows. This holds variance inflation fixed, so the gap between the two runs separates *masking* from genuine removal.
4. Repeat for 3×3 (training seed, probe seed) combinations; report medians.

Nothing retrains inside the loop. The corruption is a prompt-time edit, so a whole sweep takes seconds against cached ensembles.

A closed form for the flip. AUC is not differentiable through the sampling, which is why a REINFORCE estimator is the usual choice. For this policy it can be avoided. With $a_i = y_i\,(c_x \cdot x_i)$

over the forget slice, the change in the read-out has moments

$$\mathbb{E}[\Delta\hat{y}] = \frac{-2\theta}{N+1}\sum_i a_i, \qquad \mathrm{Var}[\Delta\hat{y}] = \frac{4\theta(1-\theta)}{(N+1)^2}\sum_i a_i^2, \tag{3}$$

exactly, because $(1-2B)^2 = 1$ leaves the label–label block of the context vector unchanged. Substituting (3) into the Gaussian AUC makes $\mathrm{AUC}(\theta)$ a differentiable function of θ. Monte Carlo at 4×10^5 trials per point confirms both moments to 3–4 significant figures.

Algorithm 2 — learning the corruption (Stage 2)

The stated objective is $\min_\theta \mathrm{AUC}(\theta) + \lambda\|\theta\|$. Minimising AUC overshoots. Past $\frac{1}{2}$ the attack inverts, and 0.30 is as good an adversary as 0.70. We minimise distance from chance instead:

$$\min_\theta \left(\mathrm{AUC}(\theta) - \tfrac{1}{2}\right)^2 + \lambda\|\theta\|.$$

1. Initialise θ. **2.** Compute $\mathrm{AUC}(\theta)$ from (3). **3.** Backpropagate, take an Adam step. **4.** Repeat for 400 steps. **5.** Report AUC and ε as measured by Algorithm 1, not the closed-form value.

Settings. Complete, for both datasets.

Data — synthetic		
input dimension	D	4
context length	N	31
groups		z_1, z_2, z_3; forget z_3
eigenbasis		identity, shared across groups
Λ_{z_1} eigenvalues	PR = 1.905	0.70, 0.15, 0.10, 0.05
Λ_{z_2} eigenvalues	PR = 2.381	0.60, 0.20, 0.10, 0.10
Λ_{z_3} eigenvalues	PR = 3.333	0.40, 0.30, 0.20, 0.10
covariances		trace-normalised, $\operatorname{tr} \Lambda = 1$
labels		$y = \omega^\top x$, ω per sequence, shared across groups
data seed		0
Data — MNIST		
input source		MNIST images, PCA on pooled set to $D = 4$
groups		digits $1, 3, 8$; forget digit 8
participation ratios		2.44, 3.29, 2.93
centring		per class; each bank trace-normalised
bank sizes		$\approx$ 6.7k, 6.1k, 5.9k vectors
labels		$y = \operatorname{sign}(\beta^\top x)$, β per sequence, shared
Probe (both)		
probe points	P	64
context token counts		10 / 10 / 11, forget group last
query		drawn from the forget group
probe seeds		777, 778, 779
Training (both)		
architectures		ATTN-S (factored), ATTN-M (merged)
shadow models per hypothesis	S	512
batch per shadow		8
steps		6000
optimiser		SGD, momentum 0.9
learning rate		0.005
gradient clipping		5.0, per shadow
initialisation scale		0.05
training seeds		1, 2, 3
Evaluation (both)		
seed combinations		$3 \times 3 = 9$; all values are medians
bootstrap resamples		200
shared-noise repeats		16
C1, C2 grid		21 points, $\sigma^2 \in [0, 100]$, log-spaced
C3 grid		9 points, $\sigma^2 \in [0, 10]$
flip grid		19 points, $t \in [0, 1.25]$
bern grid		21 points, $\theta \in [0, 1]$
whiten grid		9 points, mixture $\in [0, 1]$

Table 1: Experimental settings. Everything except the input source and the label rule is the same across the two datasets, so the comparison in Section 6 changes one thing at a time.

3 Control: does the attack measure anything?

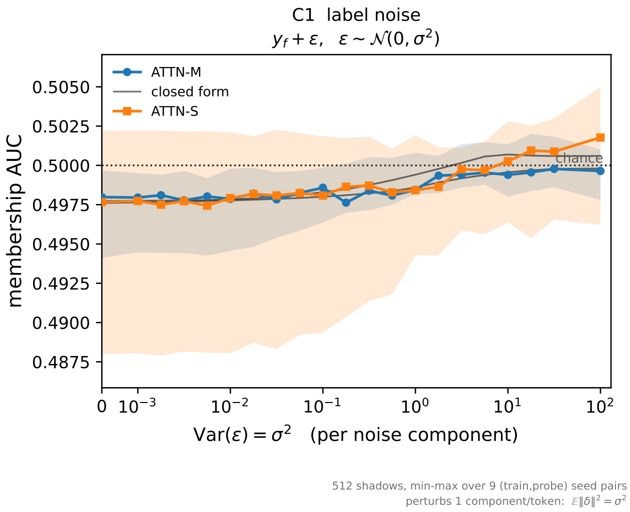


Figure 1: Null control — all three groups given the same covariance.

Give every group the same spectrum and the same eigenbasis. Then M_{full} and M_{orac} trained on the same distribution. There is no membership information to find, so the pipeline must return chance.

It does: median AUC $= 0.498$ for both architectures across the whole grid, spanning 0.4976–0.4998 and 0.4974–0.5018. The real experiments below sit at 0.535 and 0.625, far outside that band. Whatever those numbers are, they are not an artifact of the attack code.

4 The setup, at a glance

The whole setup on one page, before the results.

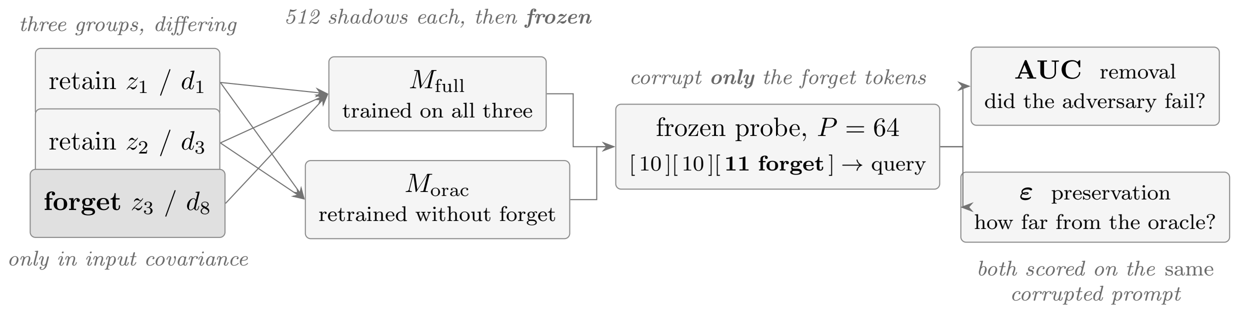


Figure 2: The pipeline. Groups differ only in the shape of their input covariance. All share one labelling rule. Two shadow ensembles are trained once and frozen: M_{full} has seen the forget group, M_{orac} has not. At evaluation the forget tokens of a frozen probe are corrupted, and both ensembles are scored on that same corrupted prompt, so detecting the edit cannot be mistaken for detecting membership. Each sweep point gives a removal number (AUC) and a preservation number (ε). Nothing retrains inside the loop.

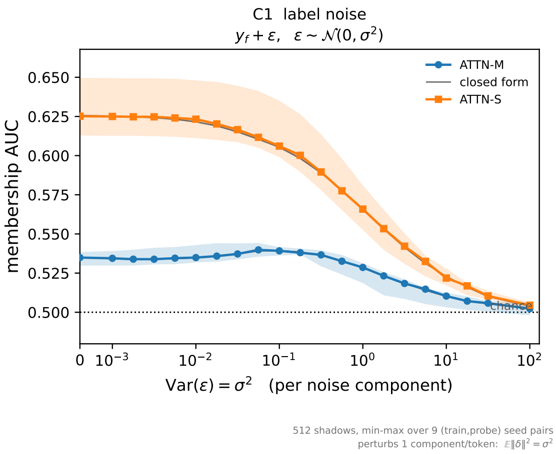
Reading the numbers.

- AUC $= \frac{1}{2}$ is the *goal* — the adversary guessing. The baseline sits *above* $\frac{1}{2}$ (a working attack) and unlearning means driving it *down* to $\frac{1}{2}$. Overshooting below $\frac{1}{2}$ is not success either: the attack has merely inverted, and $|\mathrm{AUC} - \frac{1}{2}|$ remains the signal.
- ε small is good. It is a KL against the retrain oracle, so $\varepsilon \to 0$ means "indistinguishable from

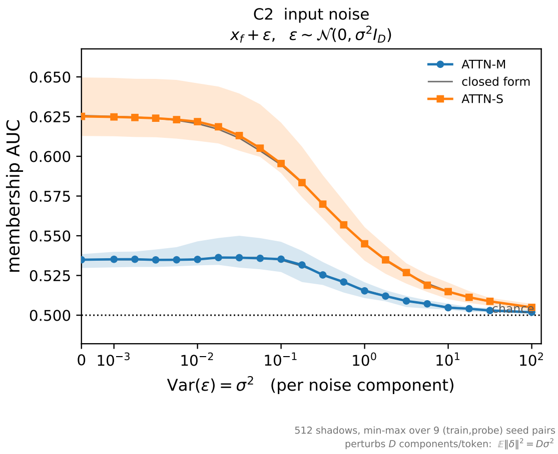
having retrained properly”.

- **Cost ratio** = ε at the closest approach to chance, divided by ε at its own minimum. It measures how far apart those two operating points are. A ratio near 1 would mean a clean operating point exists.
- Every value is a median over nine (training seed, probe seed) combinations; bands are min–max across those nine.

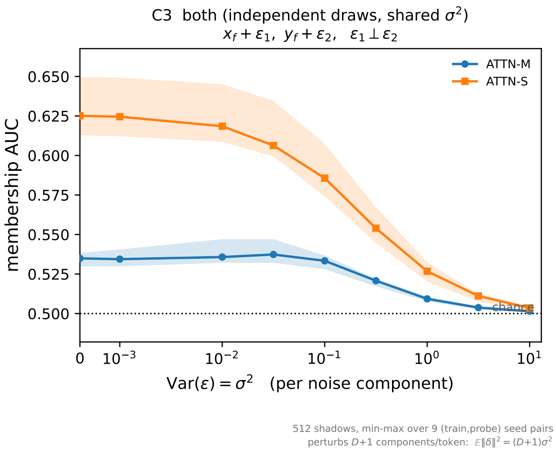
5 Experiment 1 — hand-designed corruption (synthetic)



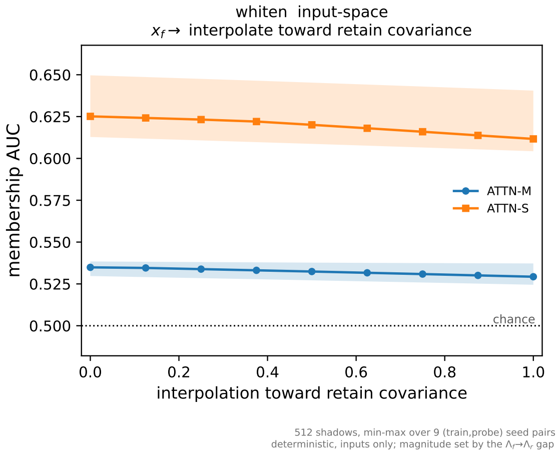
(a) C1 — label noise



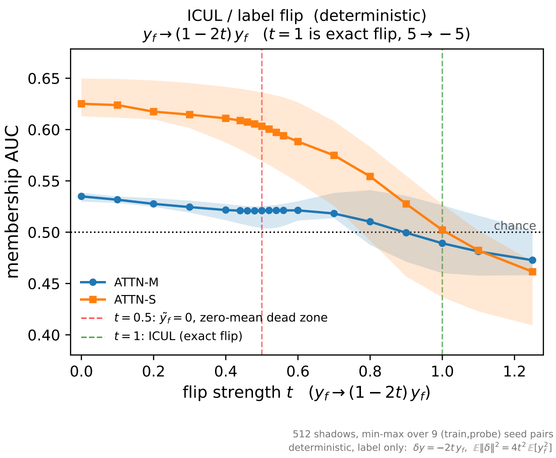
(b) C2 — input noise



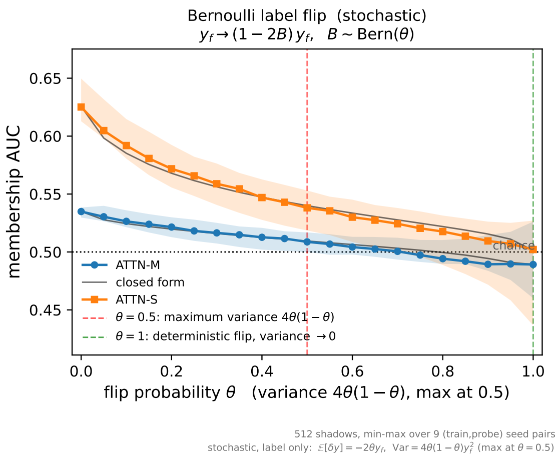
(c) C3 — both



(d) whiten — covariance matching



(e) flip — deterministic (mean only)



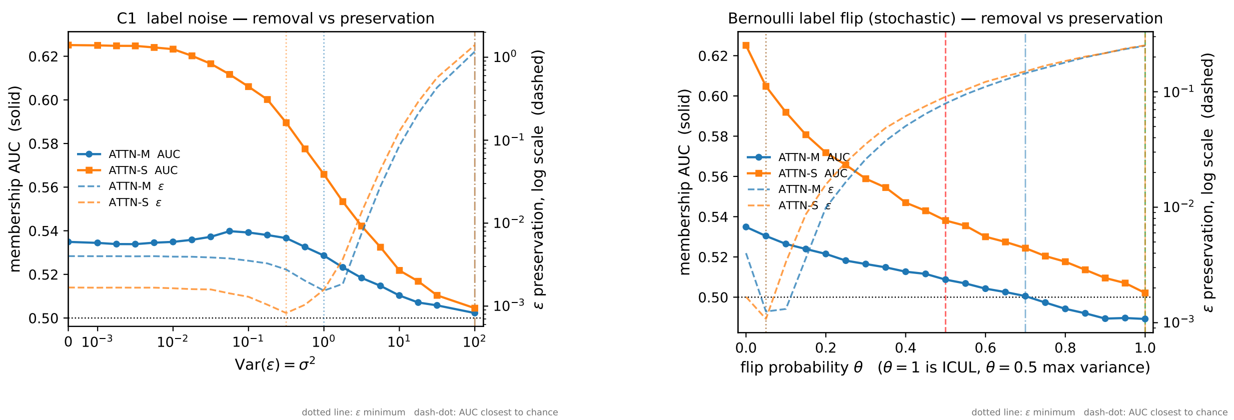
(f) bern — stochastic (mean + variance)

Figure 3: All six corruption arms, synthetic task. Membership AUC against corruption strength. Bands give the min–max over nine seed combinations. Thin line: closed-form prediction. Dotted: chance. In the bottom row, `flip` and `bern` both fall through chance, while none of the four arms above them reaches it.

At zero corruption the adversary achieves AUC = 0.535 (ATTN-M) and 0.625 (ATTN-S); per-training-seed medians span only 0.5332–0.5352 and 0.6223–0.6275, so the baselines are systematic, not sampling noise. AUC falls monotonically toward chance, reaching 0.502 and 0.504 at $\sigma^2 = 100$. The closed form tracks the Gaussian arms to 6.4×10^{-4} mean absolute error (max 2.9×10^{-3}); the Bernoulli arm is looser (3.2×10^{-3} mean, 1.7×10^{-2} max) because its mean shift sums only $n_f = 11$ Bernoulli terms and is visibly non-Gaussian near $\theta = 1$.

		best preservation		closest to chance	
arm	arch	$\varepsilon_{\min}$	at	AUC (ε)	cost ratio
C1	ATTN-M	0.00154	$\sigma^2\!=\!1.00$	0.5023 (1.170)	762×
C1	ATTN-S	0.00082	$\sigma^2\!=\!0.32$	0.5045 (1.402)	1699×
C2	ATTN-M	0.00116	$\sigma^2\!=\!0.18$	0.5019 (1.347)	1165×
C2	ATTN-S	0.00088	$\sigma^2\!=\!0.10$	0.5049 (1.435)	1630×
C3	ATTN-M	0.00158	$\sigma^2\!=\!0.10$	0.5015 (1.344)	848×
C3	ATTN-S	0.00102	$\sigma^2\!=\!0.10$	0.5035 (1.405)	1379×
flip	ATTN-M	0.00157	$t\!=\!0.10$	0.4994 (0.209)	133×
flip	ATTN-S	0.00139	$t\!=\!0.10$	0.5021 (0.252)	181×
bern	ATTN-M	0.00125	$\theta\!=\!0.05$	0.5005 (0.145)	**116**×
bern	ATTN-S	0.00108	$\theta\!=\!0.05$	0.5021 (0.252)	**234**×
whiten	ATTN-M	0.00134	$m\!=\!0.75$	0.5293 (0.002)	never reaches
whiten	ATTN-S	0.00133	$m\!=\!0.25$	0.6117 (0.004)	never reaches

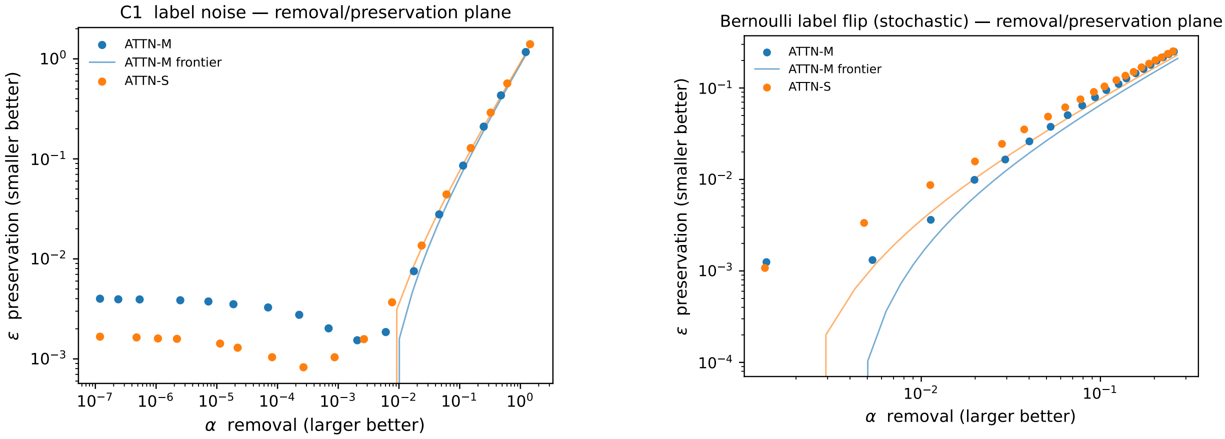
Table 2: All six arms on identical data. "Cost ratio" is ε at the closest approach to chance divided by $\varepsilon_{\min}$ — the preservation bill for defeating the adversary. The "at" column gives the corruption strength at which ε bottoms out, *in each arm's own parameter*: noise variance σ^2, deterministic flip strength t, flip probability θ, or whitening mixture m. Those are not on a common scale and should not be compared across blocks.



(a) C1 — AUC and ε against σ^2

(b) bern — AUC and ε against θ

Figure 4: The tradeoff. AUC (solid, left axis) falls toward chance while ε (dashed, right axis, log scale) climbs three decades. Vertical markers show the ε minimum and the closest approach to chance. The gap between those two markers is the cost ratio in Table 2, and it is narrower for `bern`.



(a) C1 — Gaussian noise

(b) bern — stochastic flip

Figure 5: The removal/preservation plane. Each point is one corruption strength: removal α rightward, preservation error ε upward. A usable unlearning method would reach the bottom-right. Both arms turn sharply upward instead. Past a threshold, further removal is bought almost entirely with preservation error.

Result 1. No arm has a clean operating point. Reaching chance costs two to three orders of magnitude in preservation error. Figure 5 shows the shape: the frontier is flat while removal is cheap, then turns almost vertical.

Result 2. Most of the apparent progress is masking, not removal. The shared-noise control attributes 100%, 84% and 87% of the movement at $\sigma^2 = 0.32, 0.56, 1$ to variance inflation hiding the signal. Only at $\sigma^2 = 100$ does that fall to 41%. Below $\sigma^2 \approx 0.1$ the decomposition is undefined, because AUC does not move toward chance there at all. No claim here rests on that region.

Result 3. The arms are not equally bad. They separate by which moment they move. The Gaussian arms never go below 0.501. The deterministic flip falls through chance to 0.473 and 0.461, as the Φ argument predicts. Whitening barely moves at all ($0.535 \to 0.529$). The stochastic flip has both channels and is the cheapest arm in the table by a factor of about seven.

6 Experiment 2 — replication on real data

How MNIST enters the experiment

MNIST supplies the input distributions, not the task. This is not digit classification. The construction is stated in full below because it is easy to assume otherwise.

Three groups. Take digits 1, 3 and 8; PCA the *pooled* set down to $D = 4$ so all three share one feature space; split back by digit; centre each class and normalise it so $\mathrm{tr}(\mathrm{Cov}) = 1$. The result is three banks of 4-dimensional vectors, roughly 6.7k, 6.1k and 5.9k rows, one per image. A group is a cloud of thousands of points. Because the traces are equal, the clouds differ only in covariance shape, summarised by the participation ratio $\mathrm{PR} = (\mathrm{tr}\,\Lambda)^2 / \mathrm{tr}(\Lambda^2)$: 2.44, 3.29, 2.93. Retain $= \{d_1, d_3\}$, forget $= d_8$. The forget group is the middle one, not an extreme. Unlike the synthetic setup, the geometry here is whatever the data provided.

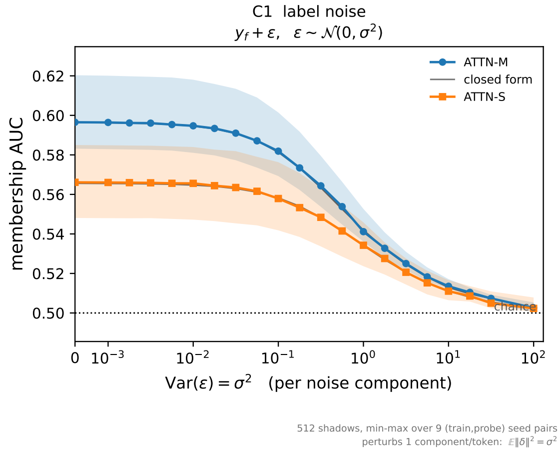
The labels come from a random plane, not from the digit. For each prompt we draw a fresh $\beta \sim \mathcal{N}(0, I_4)$ and set $y = \mathrm{sign}(\beta^\top x)$. The label alphabet is fixed at ± 1, but the labelling function changes every prompt, so the same image gets different labels in different prompts. A fixed image-to-digit rule could be memorised in the weights, leaving nothing to learn in context. A fresh β forces the model to recover the plane from the examples in front of it.

β is shared across the three groups. This constraint is essential. If each digit had its own labelling rule, corrupting the forget group would delete the answer rather than the membership evidence, and a fall in AUC would measure nothing. With β shared, all three groups obey one rule and differ only in input geometry, which is the quantity the attack is asked to detect. Sharing also keeps the retrain oracle competent on forget-group queries. It can label a d_8 point correctly, having never seen that class's geometry in training. If the digit were the label, the oracle could not answer at all, and membership inference would reduce to detecting a missing capability.

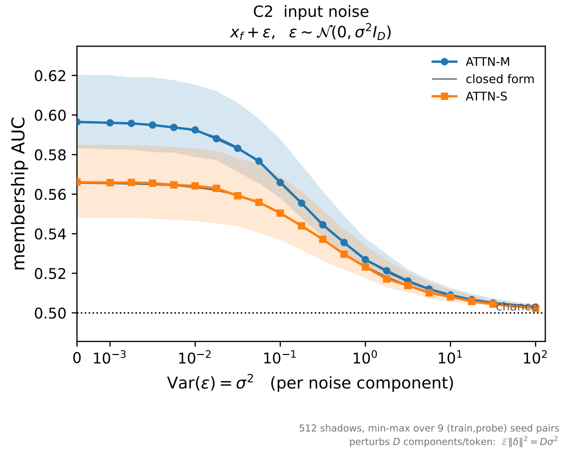
Probe layout. Each of the $P = 64$ prompts is

$$\underbrace{[\,10 \text{ from } d_1\,][\,10 \text{ from } d_3\,][\,11 \text{ from } d_8\,]}_{N=31 \text{ context tokens}} \longrightarrow \underbrace{1 \text{ more } d_8 \text{ point}}_{\text{query}}$$

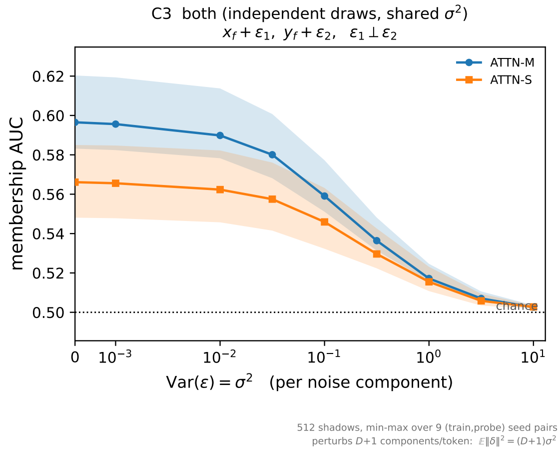
Forget tokens are placed last so they form a contiguous slice. The query is drawn from the forget group, so the model is asked about the distribution we are trying to make it forget. That is where the membership signal appears. Every corruption arm edits only those 11 d_8 context tokens, never d_1, never d_3, never the query. Everything else is unchanged from Experiment 1: the two 512-model ensembles, the attack, the sweep, and the shared-noise control.



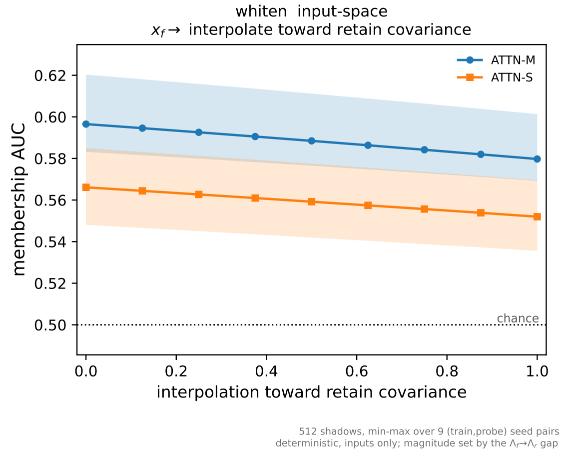
(a) C1 — label noise



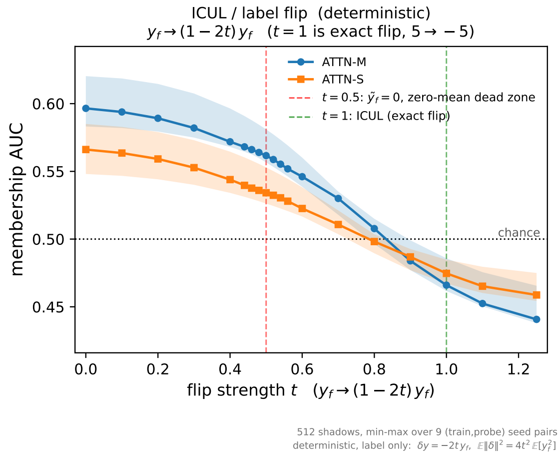
(b) C2 — input noise



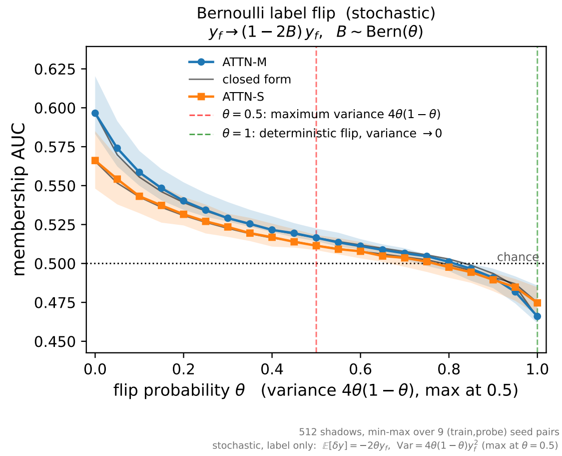
(c) C3 — both



(d) whiten — covariance matching

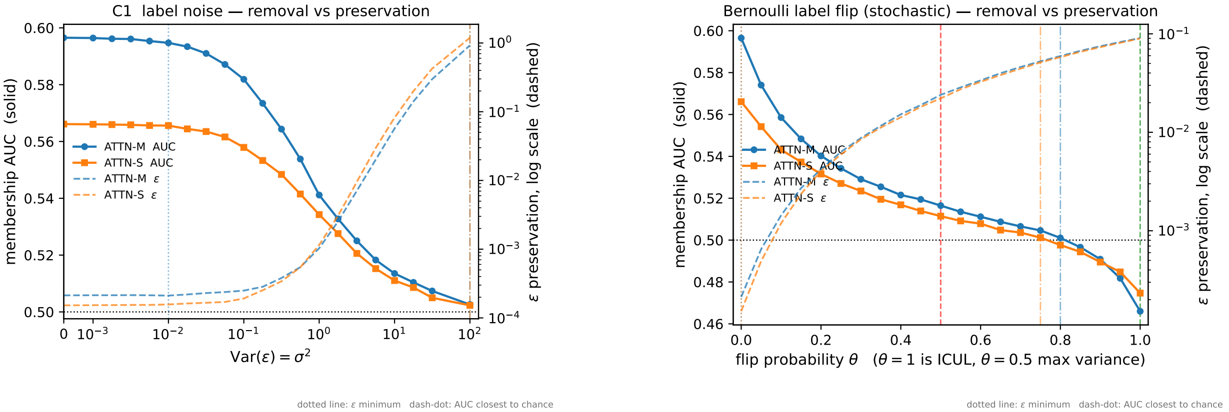


(e) flip — deterministic



(f) bern — stochastic

Figure 6: All six arms on MNIST (digits 1/3/8, forget digit 8). Same pipeline, real image features. The baselines start further from chance, so the membership signal is stronger, and the ordering of the arms is the same.



(a) C1 — Gaussian noise

(b) bern — stochastic flip

Figure 7: MNIST tradeoff. The same pair of markers as Figure 4, on real data. The separation is much wider for the Gaussian arm here than on the synthetic task. This is Result 4.

Result

The synthetic task fixes the input distribution by construction. That is what makes the closed form checkable, and also what makes the setup artificial. The question here is whether the conclusion holds when the input clouds are real: non-Gaussian, correlated, and not designed by us.

It does, and the gap widens. Baseline AUC is 0.596 (ATTN-M) and 0.566 (ATTN-S), a *stronger* membership signal than the synthetic task, and the closed form is if anything more accurate here (4.6×10^{-4} and 3.8×10^{-4} mean absolute error on C1 and C2).

		best preservation		closest to chance	
arm	arch	$\varepsilon_{\min}$	at	AUC (ε)	cost ratio
C1	ATTN-M	0.00021	$\sigma^2 = 0.01$	0.5026 (0.923)	4375×
C1	ATTN-S	0.00015	$\sigma^2 = 0$	0.5023 (1.193)	7838×
C2	ATTN-M	0.00021	$\sigma^2 = 0.001$	0.5029 (1.020)	4798×
C2	ATTN-S	0.00015	$\sigma^2 = 0$	0.5023 (1.084)	7122×
bern	ATTN-M	0.00021	$\theta = 0$	0.5010 (0.059)	**279**×
bern	ATTN-S	0.00015	$\theta = 0$	0.5012 (0.051)	**337**×

Table 3: MNIST. Columns as Table 2; again the “at” values are in each arm’s own parameter. Here every minimum sits at or beside zero corruption.

Result 4. The negative result replicates, and is worse on real data: 4375–7838× against 762–1699× synthetically.

Result 5. The flip’s advantage replicates, but its size does not transfer: 5.9–6.6× cheaper than the best Gaussian arm on the synthetic task, 15.7–21.1× on MNIST. The direction is stable. The number should not be quoted without naming the dataset.

What does not explain Result 5. The obvious hypothesis is that the advantage tracks the baseline mean gap: a larger gap needs a larger flip, which costs more. It does not. Writing the advantage as $\varepsilon_{\text{gauss}}/\varepsilon_{\text{bern}}$ at the closest approach to chance (the $\varepsilon_{\min}$ cancels), and the standardised baseline gap as $z_0 = \Phi^{-1}(\text{AUC}_0)$:

dataset	arch	z_0	$\varepsilon_{\text{gauss}}$ / $\varepsilon_{\text{bern}}$	advantage
synthetic	ATTN-M	−0.088	1.170/0.145	8.1×
synthetic	ATTN-S	−0.319	1.402/0.252	5.6×
MNIST	ATTN-M	−0.244	0.923/0.059	15.5×
MNIST	ATTN-S	−0.167	1.084/0.051	21.1×

Across the four cases $|z_0|$ and the advantage correlate at -0.23, so there is no relationship. The driver is in the fourth column instead. $\varepsilon_{\text{gauss}}$ is similar everywhere (0.92–1.40), while $\varepsilon_{\text{bern}}$ is three to five times smaller on MNIST than on the synthetic task. The flip does not reach chance more easily on MNIST. It reaches it more cheaply. One possible mechanism, consistent with these numbers but not tested here, is that a flip's preservation cost scales with the label second moment. That is bounded for MNIST's ± 1 labels and much larger for continuous regression targets. Either way, the moment argument predicts where the optimum sits, not what it costs.

One thing does not replicate. On the synthetic task ε is minimised at non-zero corruption, so a little corruption moves the model toward the oracle. On MNIST the minimum sits at $\sigma^2 \approx 0$ for every arm.

7 Experiment 3 — learning the corruption

Experiment 1 swept θ by hand. Algorithm 2 optimises it instead.

		closed form (Alg. 2)		REINFORCE	
dataset	arch	θ^*	AUC (ε)	θ	AUC
MNIST	ATTN-S	0.678	0.4993 (0.036)	0.703	0.4975
MNIST	ATTN-M	0.753	0.4982 (0.049)	0.737	0.4997
synthetic	ATTN-S	0.949	0.5260 (0.264)	0.622	0.5423
synthetic	ATTN-M	0.850	0.5106 (0.199)	0.016	0.5386

Table 4: Both gradient estimators. The closed form takes 1.7–2.0 s against 14.1–14.5 s for REINFORCE, so 7–9× faster, and it is deterministic.

dataset	arch	θ^*	AUC achieved	ε
MNIST	ATTN-S	0.761 ± 0.061	$\mathbf{0.4987 \pm 0.0005}$	0.053 ± 0.012
MNIST	ATTN-M	0.816 ± 0.043	$\mathbf{0.4984 \pm 0.0005}$	0.061 ± 0.008
synthetic	ATTN-S	0.895 ± 0.091	0.5074 ± 0.0143	0.185 ± 0.065
synthetic	ATTN-M	0.739 ± 0.138	0.5021 ± 0.0064	0.139 ± 0.081

Table 5: Across all nine seed combinations, mean ± s.d.

Result 6. On MNIST the learned flip reaches AUC $= 0.4987 \pm 0.0005$ at $\varepsilon \approx 0.05$. That is chance to within a twentieth of a percentage point, on every seed. Gaussian noise needs $\varepsilon \approx 1.0$ to reach the same place. Three independent routes agree: the closed form gives $\theta = 0.678/0.753$, a 41-point grid search gives 0.675/0.725, and REINFORCE lands within 0.03.

Result 7. This does not transfer to the synthetic task. There the two estimators disagree sharply, $\theta = 0.850$ against 0.016 on ATTN-M, and neither reaches chance (0.5106 and 0.5386). The closed form's Bernoulli error is 3.2×10^{-3} there against 1.1×10^{-3} on MNIST. Result 9 gives the reason, which is structural rather than numerical.

Result 8, and a closed form for θ^*. The optimum flips 68–82% of forget labels. This is not a small perturbation. It is the mean-compensating solution. Setting the corrupted mean gap to zero, $\delta_0 + \mathbb{E}[\Delta\hat{y}] = 0$ with $\mathbb{E}[\Delta\hat{y}]$ from (3), gives

$$\theta^* = \frac{(N+1)\,\delta_0}{2\sum_i a_i}, \tag{4}$$

where δ_0 is the baseline mean gap. Equation (4) has no variance term. The extra spread $4\theta(1-\theta)\sum a_i^2$ sits in the denominator of the AUC and cannot move the point where the numerator vanishes.

This gives a prediction we can test on data already collected. The deterministic `flip` at strength t has the same mean shift and no variance, so its AUC must cross $\frac{1}{2}$ at exactly $t = \theta^*$. Reading that crossing off the `flip` sweep predicts the learned θ^* with no fitting:

dataset	arch	`flip` crosses $\frac{1}{2}$ at	`bern` crosses at	learned θ^*
MNIST	ATTN-S	0.785	0.767	0.761 ± 0.061
MNIST	ATTN-M	0.833	0.812	0.816 ± 0.043
synthetic	ATTN-M	0.895	0.708	0.739 ± 0.138
synthetic	ATTN-S	**1.011**	never	0.895 ± 0.091

Table 6: Predicted against realised flip fraction. On MNIST the three columns agree to within 0.02, so the closed form predicts the learned policy. On the synthetic task they do not.

Result 9: why the synthetic task fails. The last row of Table 6 explains Result 7. Cancelling the baseline gap on synthetic ATTN-S requires $t = 1.011$. But θ is a probability, so $\theta \leq 1$. The required mean cancellation is outside the range a label flip can deliver, so `bern` cannot reach chance there at all. It never does, bottoming out at 0.5021. The optimiser returning $\theta^* = 0.895 \pm 0.091$ is not converging badly. It is saturating against a bound. Synthetic ATTN-M sits just inside the feasible region, and is correspondingly noisy.

This also explains the estimator disagreement. Pushing θ^* toward 1 drives the flip's mean shift, a sum of $n_f = 11$ Bernoulli terms, into its degenerate corner, where the variance vanishes and the standardised third moment diverges. At the fitted optima the skew of that sum is -0.37 and -0.49 on MNIST, against -0.33 and -0.78 on synthetic. On the worst synthetic seed ($\theta = 0.964$) it reaches -1.50, with excess kurtosis $+2.07$. So the Gaussian AUC behind (3) is weakest exactly where the synthetic optimum is forced to sit. These are equal-weight values. Unequal a_i make the skew larger, and the synthetic task's continuous labels give a much wider spread of a_i than MNIST's ± 1.

8 Limitations

Scope. This is group-level unlearning. The adversary detects whether a distribution was in training, not whether a particular example was. Two datasets, one architecture family, and only the scalar policy. The conditional policy $p_\theta(\cdot \mid x_f, y_f)$ is implemented but not reported.

Where the method stops working. Result 9 gives a hard boundary: if cancelling the baseline gap needs a flip fraction above one, no label flip can do it. Synthetic ATTN-S is on the wrong side of that boundary. The obvious next step is to polish the closed-form optimum against measured AUC, which would help where the Gaussian approximation is weak but cannot help where the optimum is infeasible.

What the moment argument does not predict. It predicts where the optimum sits, not what it costs. Result 5 shows the flip's cost advantage varies by a factor of three between datasets and does not track the baseline gap.

Code and reproduction. All code, configs and figures are at `https://github.com/manyagupta13/icl-unlearning` (branch `followup-experiments`). Every number in this report comes from `artifacts/results_regression.csv`, `artifacts/results_mnist.csv` and the Stage 2 JSONs, and each is checked against those files programmatically. The full pipeline is four commands:

```
train_ensembles.py → run_auc_sweep.py → plot_*.py → stage2_optimise.py
```

each taking `--config configs/{regression,mnist}.yaml`. Only the first needs a GPU. The correctness checks in `tests/` run on CPU; `verify_bern.py` is the one that matters for Equation (3).

A Derivation of the flip moments and of θ^*

Equation (3) is stated in Section 2 without proof. The derivation is four steps.

Step 1 — the read-out is linear in the context labels. For linear attention performing in-context regression, the prediction at the query is a weighted sum of the context labels,

$$\hat{y}_q \;=\; \frac{1}{N+1}\sum_i y_i\,(c_x\!\cdot\! x_i), \qquad c_x = M^\top x_q.$$

Write $a_i = y_i\,(c_x\!\cdot\! x_i)$ for the signed contribution of context token i, so $\hat{y}_q = \frac{1}{N+1}\sum_i a_i$. Linearity in y is the only property used below.

Step 2 — what the flip does. The policy replaces y_i by $(1-2B_i)y_i$ for i in the forget slice, with $B_i \sim \mathrm{Bern}(\theta)$ drawn independently. Token i's contribution becomes $(1-2B_i)a_i$, so the change in the prediction is

$$\Delta\hat{y} \;=\; \frac{1}{N+1}\sum_{i\in f}\big[(1-2B_i)-1\big]a_i \;=\; \frac{-2}{N+1}\sum_{i\in f} B_i\,a_i.$$

A weighted sum of independent Bernoulli variables.

Step 3 — its moments. With $\mathbb{E}[B_i]=\theta$ and $\mathrm{Var}(B_i)=\theta(1-\theta)$, independence gives (3) immediately:

$$\mathbb{E}[\Delta\hat{y}] = \frac{-2\theta}{N+1}\sum_i a_i, \qquad \mathrm{Var}[\Delta\hat{y}] = \frac{4\theta(1-\theta)}{(N+1)^2}\sum_i a_i^2.$$

Step 4 — why this is exact, not first-order. The context vector also carries a label–label block, terms in $\sum_i y_i^2$. Under the flip those become $\sum_i(1-2B_i)^2y_i^2 = \sum_i y_i^2$, because $(1-2B)^2=1$ for $B\in\{0,1\}$. The block is unchanged, so no second-order correction survives and the moments hold exactly for every θ. Compare additive noise, $y\to y+\epsilon$, which sends $\sum y^2 \to \sum y^2 + 2\sum y\epsilon + \sum \epsilon^2$ and leaves terms behind.

Consequence 1 — a differentiable AUC. Modelling the two residual populations as Gaussian, $\mathrm{AUC} = \Phi\big((\mu_1-\mu_0)/\sqrt{v_1+v_0}\big)$, and (3) supplies $\mu_1(\theta)$ and $v_1(\theta)$ in closed form. So $\mathrm{AUC}(\theta)$ is an explicit smooth function and can be differentiated directly, with no sampling and no gradient variance. That is Algorithm 2.

Consequence 2 — the optimum. Setting the corrupted mean gap to zero, $\delta_0 + \mathbb{E}[\Delta\hat{y}] = 0$, and solving gives (4), $\theta^* = (N+1)\delta_0/(2\sum_i a_i)$. The variance term does not appear. It sits in the denominator of the AUC and cannot move the point where the numerator vanishes. That is why the deterministic `flip`, which has the same mean shift and no variance, must cross $\frac{1}{2}$ at the same parameter value. Table 6 tests that prediction.

Verification. `tests/verify_bern.py` draws 4×10^5 Monte Carlo realisations per grid point and recovers both moments to 3–4 significant figures, and confirms $\sum\tilde{y}^2 - \sum y^2 = 0$ exactly.

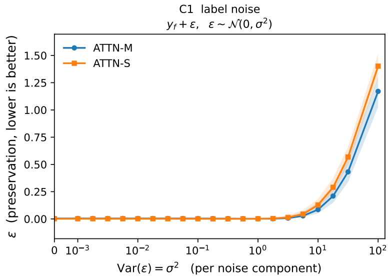
The two estimators, side by side

Closed form (Algorithm 2)	**REINFORCE (the brief)**
Uses the formula for $\mathrm{AUC}(\theta)$ above and differentiates it.	Never uses the formula. Samples masks and infers the slope from measurements.
Per step: evaluate a formula, autograd, Adam step. No model is run.	Per step: draw 32 masks, apply each, run both 512-model ensembles, weight by the score $\nabla_\theta \log p_\theta(B) = \sum_i [B_i/\theta - (1-B_i)/(1-\theta)]$.
Deterministic — identical answer every run.	Stochastic — a different answer every run; unbiased but high variance.
1.7–2.0 s.	14.1–14.5 s.
Requires the read-out to be linear in the labels. Lost if the architecture changes.	Requires only the ability to sample a corruption and measure the result. Works for any policy, any model, differentiable or not.

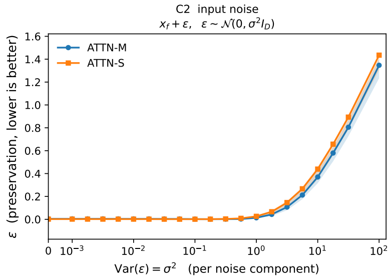
The REINFORCE derivation is correct and general. It is just unnecessary for this policy, because the model's linearity in the labels lets the expectation be computed rather than sampled. Running both checks the derivation: they optimise the same objective by independent routes, so their agreement on MNIST, within 0.03 in θ, is evidence that (3) is right.

B Preservation error against corruption strength

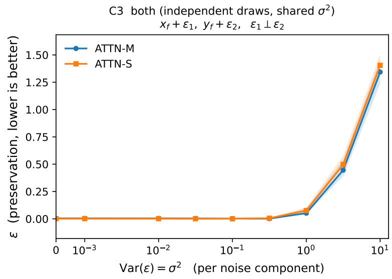
Figures 3 and 6 plot removal. These plot the other side: how ε grows as the corruption is turned up. The minimum of each curve is the $\varepsilon_{\min}$ column of Tables 2 and 3.



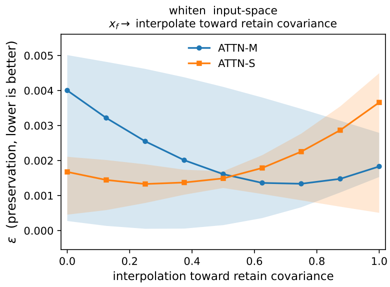
(a) C1



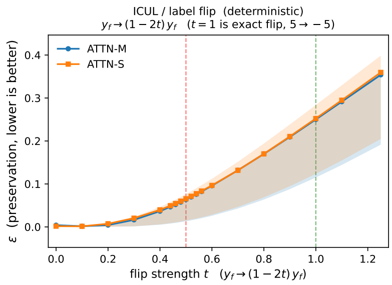
(b) C2



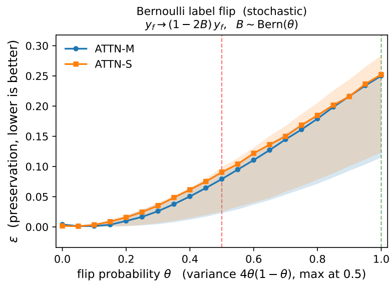
(c) C3



(d) whiten

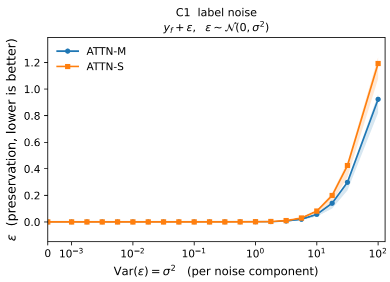


(e) flip

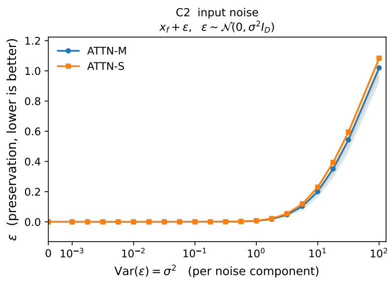


(f) bern

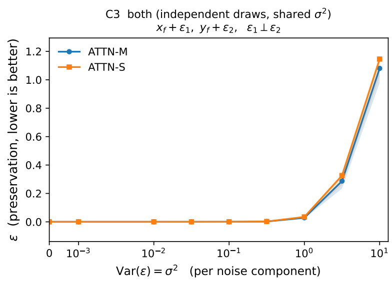
Figure 8: Preservation error, synthetic task. Several arms dip below the zero-corruption value: mild corruption moves the unlearned model toward the oracle before the cost takes over.



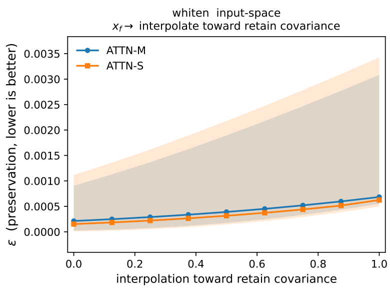
(a) C1



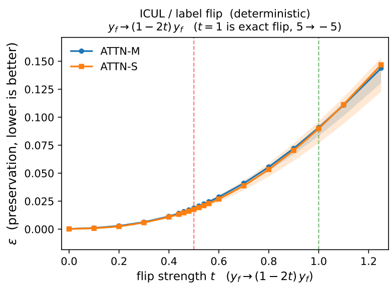
(b) C2



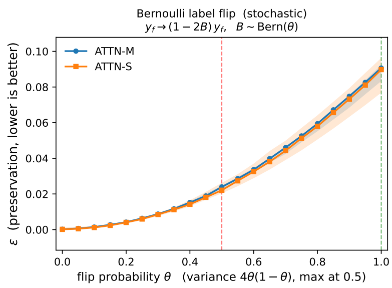
(c) C3



(d) whiten



(e) flip

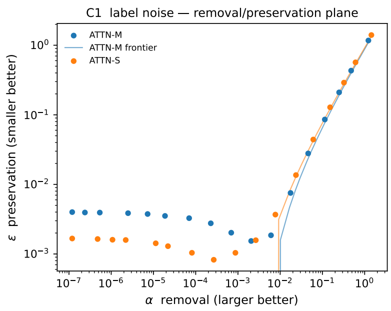


(f) bern

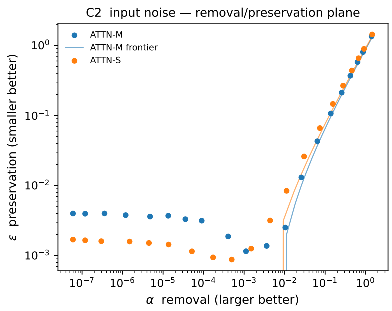
Figure 9: Preservation error, MNIST. The dip is absent. The minimum sits at zero corruption for every arm.

C Removal/preservation frontiers, all arms

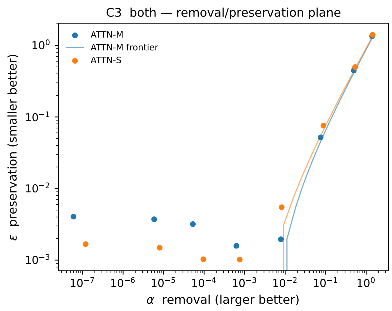
Each point is one corruption strength, plotted as removal α (rightward, larger better) against preservation error ε (upward, smaller better). A method with a clean operating point would give a frontier that extends to the right while staying low. None does.



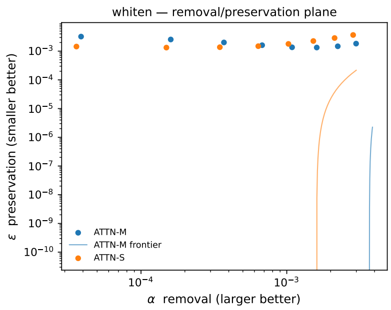
(a) C1



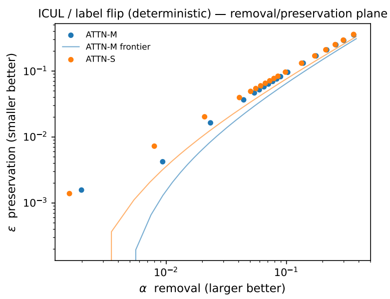
(b) C2



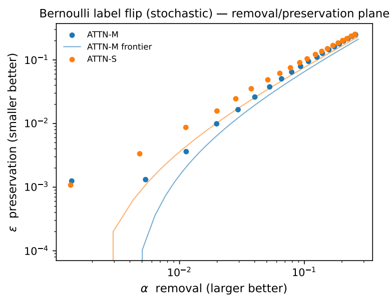
(c) C3



(d) whiten

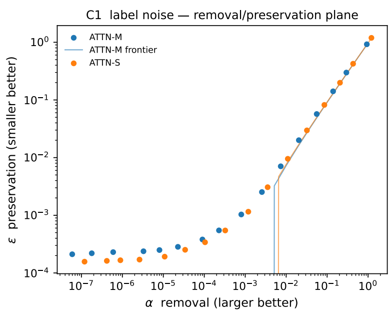


(e) flip

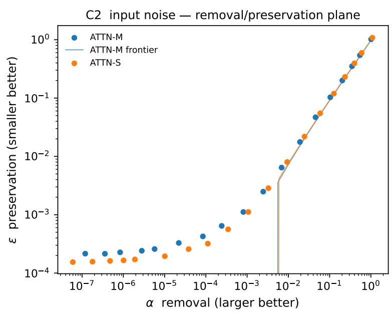


(f) bern

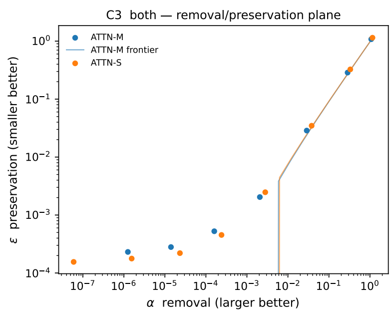
Figure 10: Frontiers, synthetic task.



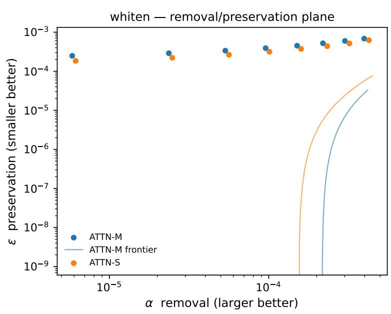
(a) C1



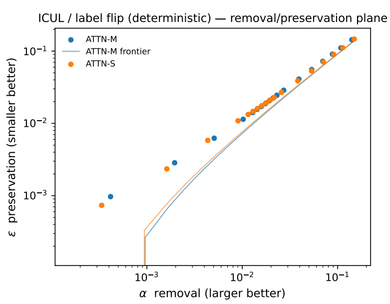
(b) C2



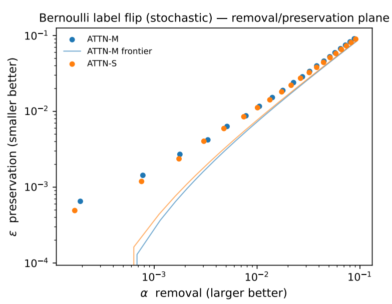
(c) C3



(d) whiten



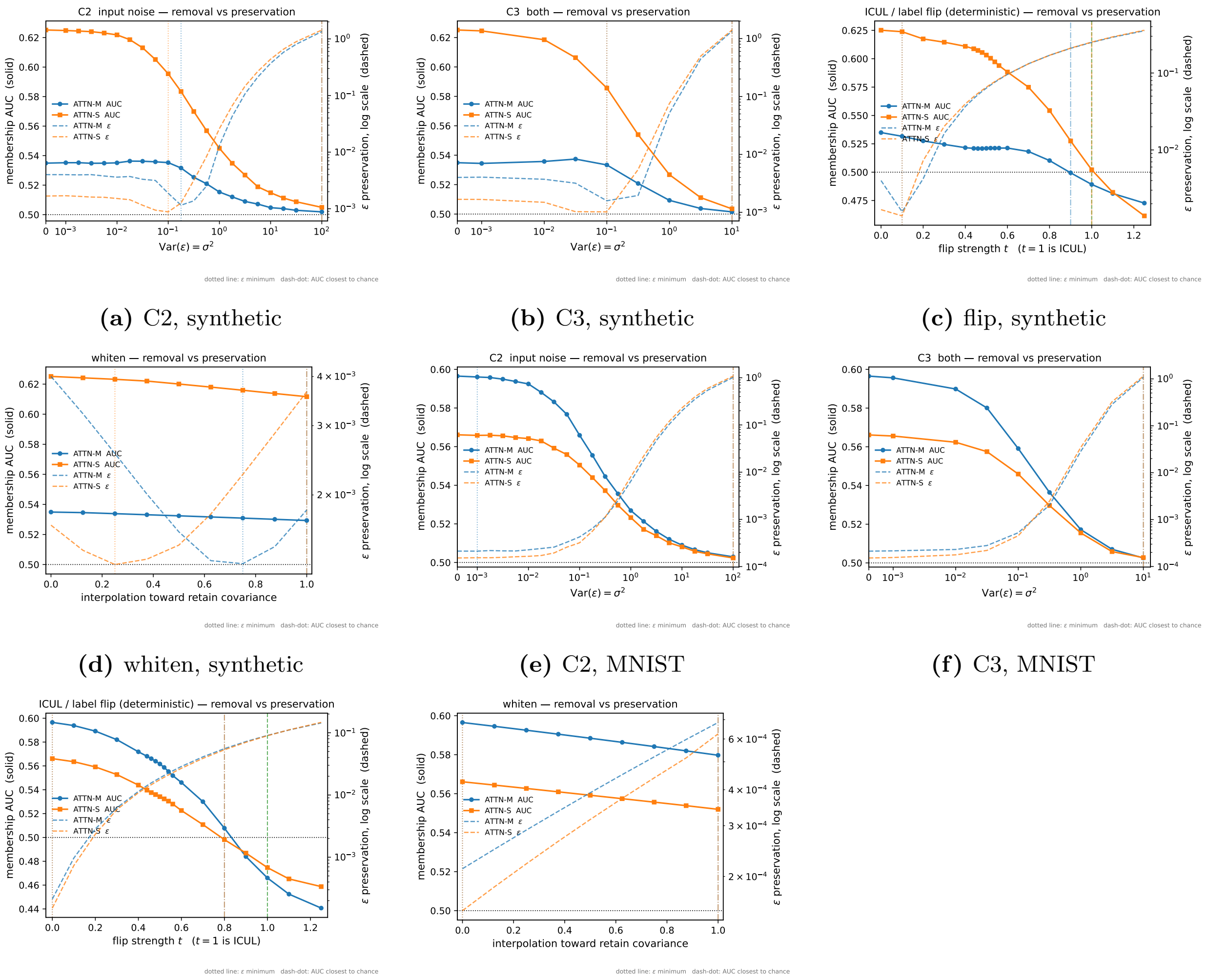
(e) flip



(f) bern

Figure 11: Frontiers, MNIST.

D Remaining tradeoff curves



(a) C2, synthetic **(b)** C3, synthetic **(c)** flip, synthetic

(d) whiten, synthetic **(e)** C2, MNIST **(f)** C3, MNIST

(g) flip, MNIST **(h)** whiten, MNIST

Figure 12: Tradeoff curves for the arms not shown in Figures 4 and 7. AUC solid on the left axis, ε dashed on the right.